# Rapport de mise en place et d'évaluation du système RAG

## Assistant IA NBA SportSee

SportSee accompagne des clubs de basket et développe un assistant IA destiné aux entraîneurs, analystes et préparateurs physiques. Le prototype répondait à partir d'archives textuelles (RAG), mais pas aux questions chiffrées dont les coachs ont besoin. Il a été évalué, enrichi d'une base de statistiques interrogée en SQL, rendu observable avec Logfire, puis évalué à nouveau.

- Questions chiffrées : c'est le gain principal. Les scores RAGAS passent de 0–0,50 à 0,875–1,00 grâce à l'outil SQL.
- Routage : les 8 questions de test sont envoyées vers la bonne source, sans aucun échec.
- Fiabilité : le système n'invente pas de chiffres et signale clairement ce qui n'est pas dans les données.
- Limites : un jeu de test réduit, une recherche documentaire perfectible et enrichissement des données.

### Contexte et objectifs

SportSee travaille avec des clubs de basket pour valoriser leurs archives, leurs rapports d'analyse et leurs données de matchs. L'objectif est d'aider les entraîneurs, analystes et préparateurs physiques à trouver rapidement les informations utiles pour préparer les entraînements et les matchs, et suivre la progression des joueurs. Le prototype de Sarah combine une interface Streamlit, un index vectoriel FAISS et un modèle Mistral : il retrouve les passages utiles dans les archives textuelles, puis rédige une réponse. Ses résultats étaient encourageants sur ces textes, mais il ne savait pas répondre aux questions chiffrées précises des coachs.
La mission s'est déroulée en trois étapes :

- Évaluer : mettre en place l'évaluation RAGAS, fiabiliser la préparation des données avec Pydantic et tracer le pipeline avec Logfire.
- Enrichir : charger le fichier Excel des statistiques dans une base SQL et créer un outil SQL utilisable par l'assistant.
- Réévaluer : mesurer l'apport du SQL, étendre les tests aux questions mixtes et analyser les limites.

Deux sources de données : les archives textuelles (quatre discussions Reddit en PDF pour la recherche documentaire et le fichier regular NBA.xlsx (569 joueurs, 30 équipes, statistiques de la saison) pour la base SQL.

## Environnement et organisation du projet

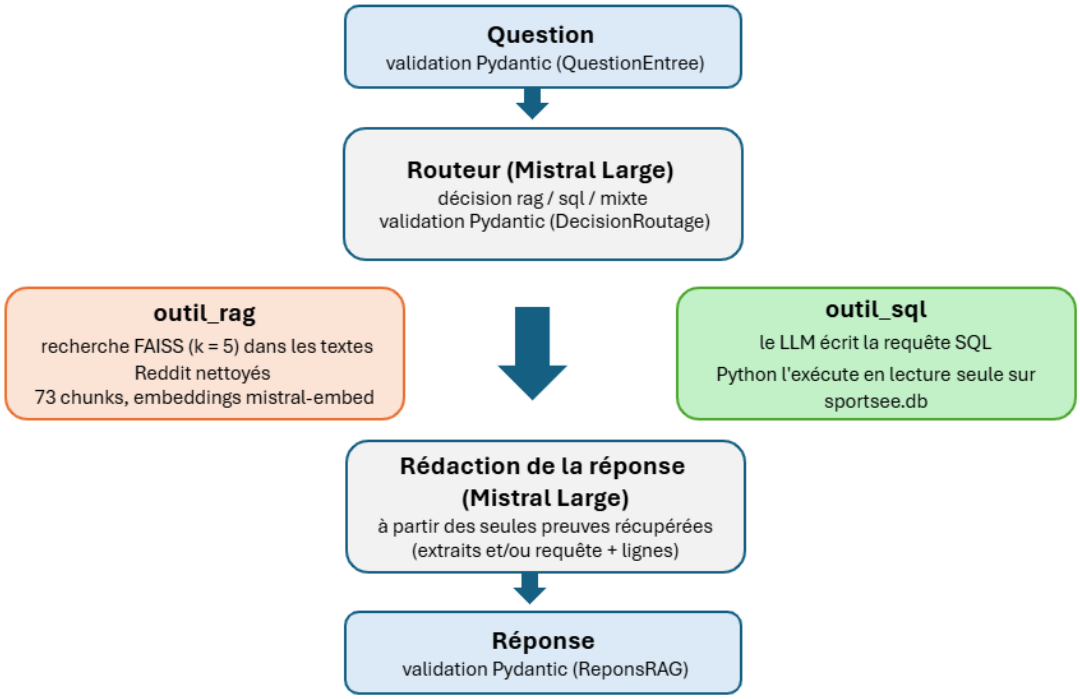
L'environnement est reproductible : requirements.txt.
Les clés d'API sont lues depuis un fichier .env.
Le README décrit l'installation, le lancement de l'application et des évaluations.
Les scripts sont organisés par rôle :

- Préparation des données : data_prep/ (extraction OCR, nettoyage), load_excel_to_db.py, audit_chunks.py.
- Recherche documentaire : utils/vector_store.py (découpage, embeddings, index FAISS, recherche) et indexer.py.
- Assistant IA : routeur_rag_sql.py, sql_tool.py et l'interface MistralChat.py.
- Validation et suivi : models.py (Pydantic) et logfire_setup.py.
- Évaluation : evaluate_ragas.py, evaluate_ragas_apres.py, questions_test.json.

## Architecture du système

Chaque question suit le même parcours, de la validation de l'entrée à la validation de la réponse et chaque étape est tracée dans Pydantic Logfire.
Routage à étapes fixes : le routeur lit la question et choisit la source, la recherche documentaire pour une question de contexte, l'outil SQL pour une question chiffrée, les deux pour une question mixte. Le modèle rédige ensuite la réponse à partir des seules

informations récupérées.




## Données et recherche documentaire

Le prototype indexait les PDF Reddit et le fichier Excel tels quels : les extraits mélangeaient le texte des discussions, le bruit de la page web et des lignes de tableau. La première étape a donc consisté à nettoyer les données avant de mesurer le système.

**Préparation des textes**

- Extraction : les fils Reddit sont des pages web scannées ; le texte est extrait par OCR.
- Nettoyage : le bruit d'interface (boutons, pseudos, dates, votes, publicités) est retiré. Le texte perd 36 % de son volume et les 581 lignes de bruit repérées disparaissent.
- Validation : chaque texte nettoyé est contrôlé par Pydantic : longueur minimale et aucun bruit restant.

**Mise à jour de la recherche documentaire**

- Source de l'index : l'index est désormais construit à partir des seuls textes nettoyés. L'Excel en est retiré : les statistiques sont servies par la base SQL.
- Découpage : extraits de 1 500 caractères avec un chevauchement de 150, soit 73 extraits.
- Vectorisation : embeddings mistral-embed (1 024 dimensions). Chaque extrait et chaque vecteur sont validés par Pydantic avant d'entrer dans l'index : texte non vide, bonne dimension, pas de valeur manquante ou infinie.
- Recherche : index FAISS en similarité cosinus ; les 5 extraits les plus proches de la question sont transmis au modèle.

- Flux de l'assistant : la question posée, la décision du routeur et la réponse sont elles aussi validées par Pydantic. Une donnée invalide arrête le traitement avec un message clair.

**Effet sur l'index**

Le script audit_chunks.py a été lancé sur l'index du prototype de base puis sur l'index reconstruit:

- Taille : 302 extraits avant (100 issus des PDF, 175 de l'Excel, 27 d'une feuille vide), 73 après, tous issus des discussions nettoyées.
- Bruit d'interface : 95 extraits pollués sur 100 avant, aucun après.
- Extraits Excel : avant, 142 des 143 extraits de la feuille de statistiques n'avaient pas les noms de colonnes et tous contenaient des cases vides, ce qui rendait les chiffres inutilisables ; ils sont désormais remplacés par la base SQL.
- Doublons : 16 avant, aucun après.

## Base de données et outil SQL

Le script load_excel_to_db.py crée la base SQLite sportsee.db à partir de l'Excel. Il définit d'abord les tables et leurs clés, puis lit les feuilles, valide chaque ligne avec Pydantic et insère les données. Les colonnes de la table des statistiques sont générées à partir du modèle Pydantic des joueurs : la base et la validation partagent ainsi toujours les mêmes colonnes. La base est reconstruite à chaque chargement, ce qui évite tout doublon.

**Schéma relationnel :**

- teams (30 lignes) : le code de l'équipe (clé primaire) et son nom complet.
- players (569 lignes) : l'identité du joueur : nom, âge, avec player_id comme clé primaire et team_code comme clé étrangère vers teams.
- reports (1 ligne) : la source des statistiques : saison, type de saison et fichier importé, avec report_id comme clé primaire.
- stats (569 lignes) : les 42 indicateurs d'un joueur pour un rapport donné. Elle est reliée à players et à reports par deux clés étrangères. Un joueur n'a qu'une ligne par rapport, et supprimer un rapport supprime ses statistiques.
- matches (vide) : prévue pour les résultats match par match (équipes à domicile et à l'extérieur, scores).

Séparer l'identité du joueur de ses statistiques permettra d'ajouter une nouvelle saison sans dupliquer les joueurs. Les colonnes de jointure sont indexées, et les totaux par équipe ne sont pas stockés : ils se recalculent à partir des joueurs, ce qui garantit leur cohérence.

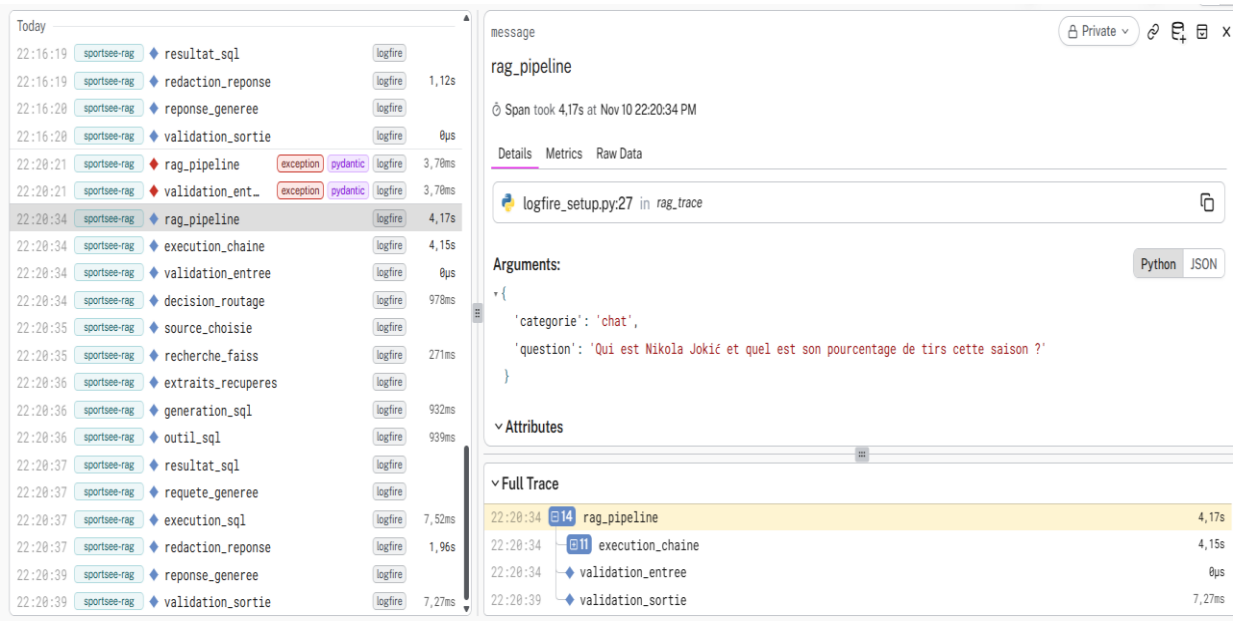
**Fonctionnement de l'outil SQL :** Le modèle écrit une requête à partir du schéma de la base, du dictionnaire des données et d'exemples de requêtes types (few-shot). Python exécute ensuite la requête en lecture seule : la réponse s'appuie ainsi sur des chiffres réellement lus dans la base. Un contrôle qualité du fichier Excel (saison, définitions, colonne mal convertie) a été mené et ses résultats sont intégrés au prompt SQL avec des

conventions pour guider le LLM dans l'écriture des requêtes face aux formulations ambiguës.

**Interface :** L'application Streamlit (MistralChat.py) est branchée sur le routeur : chaque question profite de la recherche documentaire, de l'outil SQL, des validations et du suivi Logfire. Une question invalide reçoit un message invitant à la reformuler.

## Observabilité avec Logfire

Chaque question est tracée de bout en bout, avec la durée de chaque étape : validation de l'entrée, décision de routage, recherche FAISS, génération puis exécution de la requête SQL, rédaction de la réponse et validation de la sortie.




## Méthode d'évaluation

Le jeu de test, stocké dans questions_test.json, compte 8 questions réparties en quatre catégories : simples, bruitées, complexes et mixtes. Chaque question précise la source attendue et une réponse de référence (ground truth) tirée des données.

Chaque catégorie correspond à un usage réel :

- Simples : les questions de contexte
- Complexes : les classements et comparaisons chiffrées
- Bruitées : les questions tapées vite, sans accents, avec abréviations ou fautes
- Mixtes : les questions qui mêlent contexte et chiffres et obligent le système à combiner ses deux sources.

**Protocole :**

- Avant : le RAG, données nettoyées, sur les 6 questions simples, complexes et bruitées.
- Après : le système RAG + SQL, sur les mêmes questions et les 2 questions mixtes.

Les deux versions partagent les mêmes données nettoyées, le même index, le même modèle. La comparaison se fait donc à qualité de données égale, et l'écart mesuré s'attribue à l'ajout du SQL.

**Métriques** :

- faithfulness : la réponse s'appuie-t-elle uniquement sur les contextes récupérés ?
- answer relevancy : répond-elle à la question posée ?
- context precision : les contextes récupérés sont-ils pertinents ?
- context recall : contiennent-ils toutes les informations nécessaires ?
- taux de bon routage : part des questions de test pour lesquelles le routeur a choisi la bonne source.

## Résultats

Résultat RAGAS avant :

```
Scores par question
   id categorie  faithfulness  answer_relevancy  context_precision  context_recall
0  S1    simple          0.75             0.858              0.500             1.0
1  S2    simple          0.00             0.848              0.333             0.0
2  C1  complexe          1.00             0.000              0.000             0.0
3  C2  complexe          0.00             0.000              0.000             0.0
4  B1    bruité          0.80             0.680              0.887             1.0
5  B2    bruité          1.00             0.686              0.750             0.5

Moyenne par catégorie
            faithfulness  answer_relevancy  context_precision  context_recall
categorie
bruité             0.900             0.683              0.819            0.75
complexe           0.500             0.000              0.000            0.00
simple             0.375             0.853              0.417            0.50

Échecs écartés : 0/6
```

Résultat RAGAS après :

```
Scores par question
   id categorie source_detectee  routage_ok  faithfulness  answer_relevancy  context_precision  context_recall
0  S1    simple            rag        True         0.625             0.858              0.417             1.0
1  S2    simple            rag        True         0.000             0.853              0.833             0.0
2  C1  complexe            sql        True         0.750             0.926              1.000             1.0
3  C2  complexe            sql        True         1.000             0.894              1.000             1.0
4  B1    bruité            rag        True         0.500             0.680              0.887             1.0
5  B2    bruité            rag        True         1.000             0.679              0.750             1.0
6  M1     mixte          mixte        True         0.857             0.958              0.167             0.5
7  M2     mixte          mixte        True         0.833             0.746              0.167             1.0

Moyenne par catégorie
            faithfulness  answer_relevancy  context_precision  context_recall
categorie
bruité             0.750             0.680              0.819            1.00
complexe           0.875             0.910              1.000            1.00
mixte              0.845             0.852              0.167            0.75
simple             0.312             0.856              0.625            0.50

Taux de bon routage : 100 % (8/8)
Échecs écartés      : 0/8
```

- Questions chiffrées : le RAG seul ne trouvait aucun chiffre dans les discussions (pertinence, précision et rappel à 0). Avec le SQL, les quatre métriques atteignent 0,875 à 1,00 et les réponses citent les bonnes valeurs.
- Questions simples et bruitées : elles passent par la recherche documentaire dans les deux versions. Leurs scores varient d'une évaluation à l'autre, parce que le juge RAGAS et le modèle ne sont pas parfaitement déterministes. Sur 2 questions par catégorie, ces écarts ne sont pas significatifs. Les questions avec fautes retrouvent bien leurs passages (rappel de 1,00).
- Questions mixtes : les deux outils sont appelés et les réponses sont fidèles et pertinentes ; la partie documentaire reste moins précise.

**Comparaison au seuil :** Un seuil de 0,80 a été fixé pour toutes les métriques. En effet, l'assistant aide des entraîneurs et des analystes à préparer des matchs. Une réponse fausse peut donc fausser une décision, d'où l'éxigence élevée.
Après ajout du SQL, les questions complexes le dépassent sur les quatre métriques, ainsi que la fidélité et la pertinence des questions mixtes. Les écarts sont dus principalement à la recherche documentaire : fidélité et rappel des questions simples, pénalisés par la question S2 dont le passage utile n'est pas retrouvé et précision de contexte des questions mixtes, les archives Reddit décrivant peu les joueurs. La recherche est donc à améliorer.

Pour un entraîneur ou un analyste, le système est donc fiable sur les chiffres de la saison.

## Limites et recommandations

<u>Limites :</u>

- Taille du test : avec 2 questions par catégorie, les écarts ne sont pas très significatifs, seul le gain sur les questions chiffrées est net.
- Recherche documentaire : la recherche sémantique peut manquer un passage pertinent : question en français sur un corpus en anglais, faible poids des noms propres et des dates, limite à 5 extraits.
- Données : uniquement des statistiques de saison : les analyses match par match restent hors de portée.

<u>Recommandations :</u>

- Données match par match : alimenter la table des matchs déjà prévue.
- Recherche hybride : combiner FAISS et une recherche par mots-clés pour améliorer le rappel.
- Jeu de test : l'élargir à 30-50 questions validées par le métier, avec un juge d'une autre famille de modèles.

## Conclusion :

Le prototype est devenu un système hybride, mesuré et observable. Il répond désormais aux questions chiffrées avec des valeurs exactes, choisit la bonne source pour chaque question et ne dégrade pas les questions documentaires.